

Week 3

A Landscape Study of Open Source and Proprietary Tools for SBOM

1. The researchers looked at 84 different SBOM tools available on the market and they wanted to see if there were any missing features or limitations. They organized Plugfest, where they tested five of the most popular open-source SBOM tools to see if there were any compatibility issues.
2. Findings:
 - I. Many SBOM tools rely on package managers, if package managers aren't available or if unrelated packages included this leads to the generation of inaccurate SBOMS.
 - II. Different SBOM tools use different formats that make it hard to share or combine SBOMs (CycloneDX and SPDX formats).
 - III. Many SBOM tools designed for specific programming languages thus they are less versatile if a company uses different languages.

Generating SBOMs in Cyclone DX and SPDX Formats using Syft with OWASP's NodeGoat Project

Aspect	Cyclone DX	SPDX
Granularity	Includes detailed properties for each component like the type, programming language used and its exact location within the project.	Has source info field which tells us where the package was discovered. Does not have detailed properties.
Component types	Specifies the type of each component (for example if it is a library or an application).	No type field. Uses externalRefs to categorize components
Component location	Includes explicit information about location of component within project.	Does not have location of component within project.
Download location	Includes download location in the component properties, typically under the "properties" section. This specifies the source from where the component was obtained .	Includes a downloadLocation field, which explicitly specifies where the package can be downloaded from (if available). May not align with CycloneDX's definition.

CycloneDX SBOM Format

```
{ "$schema": "http://cyclonedx.org/schema/bom-1.6.schema.json",  
  "bomFormat": "CycloneDX",  
  "specVersion": "1.6",  
  "serialNumber":  
    "urn:uuid:bb33d2aa-6088-4d16-90c0-96845efa5a28",  
  "version": 1,  
  "metadata": {  
    "timestamp": "2024-11-08T00:59:14+05:00",  
    "tools": {  
      "components": [{  
        "type": "application",  
        "author": "anchore",  
        "name": "syft",  
        "version": "1.16.0"  
      }],  
      "component": { "bom-ref": "af63bd4c8601b7f1", "type": "file",  
        "name": "." }},
```

```
    "components": [  
      {  
        "bom-ref": "pkg:npm/abbrev@1.1.1?package-id=61b98598c0759a7b",  
        "type": "library",  
        ...  
        "properties": [{  
          "name": "syft:package:foundBy",  
          "value": "javascript-lock-cataloger"  
        }],  
        { "name": "syft:package:language", "value": "javascript" },  
        { "name": "syft:package:type", "value": "npm" },  
        {  
          "name": "syft:package:metadataType",  
          "value": "javascript-npm-package-lock-entry" },  
        { "name": "syft:location:0:path", "value": "\\package-lock.json" }  
      ] },...
```

SPDX SBOM Format

```
{
  "spdxVersion": "SPDX-2.3",
  "dataLicense": "CC0-1.0",
  "SPDXID": "SPDXRef-DOCUMENT",
  "name": ".",
  "documentNamespace":
    "https://anchore.com/syft/dir/d2c5caf3-e699-410e-a786-951da44
    849f7",
  "creationInfo": {
    "licenseListVersion": "3.25",
    "creators": ["Organization: Anchore, Inc", "Tool: syft-1.16.0"],
    "created": "2024-11-07T20:00:14Z"
  },...
  "packages": [{
    "name": "abbrev",
    "SPDXID": "SPDXRef-Package-npm-abbrev-61b98598c0759a7b",
    "versionInfo": "1.1.1",
    "supplier": "NOASSERTION",

    "downloadLocation": "NONE",
    "filesAnalyzed": false,
    "sourceInfo": "acquired package info from installed node module manifest file:
    \\package-lock.json",
    "licenseConcluded": "NOASSERTION",
    "licenseDeclared": "ISC",
    "copyrightText": "NOASSERTION",
    "externalRefs": [{
      "referenceCategory": "SECURITY",
      "referenceType": "cpe23Type",
      "referenceLocator": "cpe:2.3:a:abbrev:abbrev:1.1.1:*:*:*:*:*:*"
    }],{
      "referenceCategory": "PACKAGE-MANAGER",
      "referenceType": "purl",
      "referenceLocator": "pkg:npm/abbrev@1.1.1"
    }
  ]
}
```

WEEK 4

CycloneDX CLI

Key Features	Description and Formats Supported
SBOM Creation	Supports formats: CycloneDX XML, CycloneDX JSON, CycloneDX Protobuf, CycloneDX CSV, SPDX JSON
SBOM Analysis	Reporting on components that are included multiple times with different versions
Diffing	Highlights the components added, removed, or modified, between two SBOMs.
Format Conversion	Conversion is supported between CycloneDX XML, JSON, Protobuf, CSV, and SPDX JSON v2.2.
Keygen command	Generates a pair of cryptographic keys: a private key for signing and a public key for verification.
Signing	Loads the private key, reads the SBOM file, generates a signature, and appends a Signature tag to the end of the SBOM file.
Verification	Checks all digital signatures within a given SBOM. Uses the public key to verify the signatures.

CycloneDX CLI

- When the CLI is used to generate SBOMs it only specifies type as "file". It also only includes the version number and hashes of each component.
- The analyse command focuses only on identifying multiple versions of the components.
- In the diff command is primarily focused on comparing component versions.
- The Verify command works only on XML-formatted SBOMs.
- All signatures in a SBOM must pass verification for the process to succeed. Partial verification (e.g., accepting some valid signatures while reporting failed ones) is not supported.
- The CLI outputs simple pass/fail results for signature verification. No details into why verification failed.

WEEK 5

Why Johnny Can't Encrypt

Usability for Security

If people:

- Are reliably made aware of the security tasks they need to perform
- Are able to figure out how to successfully perform those tasks
- Don't make dangerous errors
- Are sufficiently comfortable with the interface to continue using it

Security and user interface design

- The unmotivated user property
- Abstraction property
- Lack of feedback property
- The barn door property
- The weakest link property

Cosign

Keyless Signing:

Uses temporary keys generated for each signing event.

These keys are linked to the signer's identity using OpenID Connect (OIDC).

The keys are discarded after signing, reducing the attack surface.

Cosign computes a cryptographic hash of the artifact to be signed.

The temporary private key signs this hash, producing a digital signature.

The signature can be: Attached to the artifact or stored separately.

Every signing event is recorded in a public transparency log.

WEEK 6

Signing in Four Public Software Package Registries

Signing Frequency: Registry-specific signing policies significantly impact signing frequency (mandatory signing leads to higher adoption).

Signature Quality: Dedicated signing tools improve signature quality (by automating processes and reducing errors.)

Signing-Related News: Does not significantly impact signing adoption.

Consistency in Signing: Once maintainers start signing a package, they are likely to continue signing.

Implications for SBOM signing

SBOMs generated with tools lacking signing capabilities (e.g. Syft, CycloneDX CLI) are likely to have lower signature quantity and quality (without integrating signing tools, the barriers to starting signing is also higher so users will be discouraged to start)

- Make signing mandatory during SBOM generation across all SBOM generation tools.

- SBOM-generating tools should partner with existing signing frameworks, such as Cosign, to streamline the signing process.

Action items

The paper examines public registries to analyze the quality and quantity of signed artifacts . Since a such a repository for SBOMs may not exist, we can instead evaluate current SBOM signing tools by comparing them to industry standards or state-of-the-art signing tools. (What can we use as these)

-Simulate supply chain vulnerabilities arising from using different signing tools or the absence of signing at some places in the supply chain (I'm still working on this).

Challenges in SBOM Adoption Due to Software Identifiers

Identifier availability:

SBOMs need identifiers available at the time of software creation (Some identifiers like CPEs are created too late to be useful to SBOMs)

Granularity Issues:

SBOMs need identifiers that distinguish between versions, patches and variants (OmniBOR provides unique hashes but lacks human readable labels like software name or version)

Lack of Central Authority:

There is no global system that ensures identifier consistency.

Source: <https://www.cisa.gov/sites/default/files/2023-10/Software-Identification-Ecosystem-Option-Analysis-508c.pdf>

Scheme	Derivation	Architecture	Defined By	Based On
CPE	Defined	Centralized	NIST	-
pURL	Defined	Federated	ECMA + Package Hosts	-
SWID	Defined	Distributed	Software Producer	-
SWHID	Inherent	Distributed	-	Artifact content
Nix	Inherent	Distributed	-	Package build inputs
OmniBOR	Inherent	Distributed	-	Artifact content and build inputs

USE CASES WITH SBOM

Traditional SBOM only records the package name and version.

It does not verify if the exact files match what was used in the build.

If an adversary replaces the package with a different version or a maliciously modified version, the SBOM would not detect it.

OmniBOR enhances the SBOM by including a cryptographic identifier (OID) for each component.

If any file in the package is modified, its hash will change, and the OID will not match.

The SBOM is now cryptographically linked to the exact build, making tampering easy to detect.

SBOM signing does not check if the actual files used in the build match the SBOM (it just checks if the sbom has been modified).

USE CASES WITH SBOM

From what I understand one use case of OmniBOR is as a software identifier in SBOMs. this is increase the precision of SBOMs, bcs if different sbom generation tools use different component identifiers like some use purl and some use cpe, this leads to different sboms generated for the same software artifact. OmniBOR ID can be the unifying metadata amongst different naming schemes

Additionally, there is no way to represent in SBOMs today the granularity of dependencies ranging from components upto the source files. OmniBOR provides this granularity from the highest level (packages) to the lowest level (source files), especially since problems such as CVEs originate at the level of source files and hence needs to be detected at this level for higher accuracy.

Additionally, tools that operate on packaged software to produce an SBOM, such as Syft [12], do a good job of scraping the information contained in a package file and producing an SBOM with the basic data elements. However, they cannot express detailed provenance and relationship information present during the build process since that information does not persist into the generated artifact. In contrast, our results demonstrate how metadata thrown off as part of an OmniBOR-enabled build augments the SBOM generation process by capturing ephemeral elements present in the build and using those elements to create additional SBOM relationships. Our approach adds a reference in the SBOM to the root of the ADG (representing the entire unit of software) capturing build-time and run-time dependency information.

OmniBOR (Universal Bill of Receipts)

- OmniBOR is designed to consistently build verifiable Artifact Dependency Graphs
- Some use cases: CVE detection, build reproducibility, and augmenting SBOM generation.
- The creation of an Artifact Dependency Graph is the basis for OmniBOR which can be used to track every software artifact incorporated into a built software product. Components:
- Artifact Identifier: It is the gitoid of an artifact. Canonical, unique, immutable
- Artifact Input Manifest: Lists the Artifact Identifier of every input used to produce an artifact. The OID can also be embedded directly into executable files or can be provided in a separate file alongside the artifact whose inputs they describe.
- Artifact Dependency Graph: Merkle tree of hashes computed recursively starting from all the input artifacts that are transformed by a build tool into the final built artifact.
- Build tools embed the OmniBOR ID (OID) into each derived artifact that is built.

Compiler based OmniBOR data generation:

- OmniBOR is integrated directly into compilers (like GCC, Clang, Rustc, Go) and other build tools (linkers, assemblers).
- Whenever a build tool opens an artifact (headings, object file, archive file) during the build process, that artifact is considered dependency input artifact. The gitoid of that artifact is recorded in input manifest.
- For every compilation and link command the tool produces artifact input manifest files, metadata files, and embeds the gitoid in output artifact.

Generation using Dynamic Tracing

- Instead of modifying compilers, OmniBOR can trace the build process externally.
- Uses bomtrace (a tool built on strace) to monitor all files accessed during compilation and linking.
- Captures input dependencies and generates IMs after the build is complete and can embed the OID into the output artifacts.
- This makes it easy to trace the exact build history of the file without extra metadata files.

Using the OmniBOR CLI from the OmniBOR-rs project (rust implementation)

This CLI facilitates the generation and management of OmniBOR identifiers and manifests.

omnibor-test

| -- config.json

| -- test.py

| -- README.md

- Generated Artifact IDs for a Single File (test.py)

`Get-ChildItem -File | ForEach-Object { omnibor id $_.Name }`

config.json => gitoid:blob:sha256:2b20d29471b63de3a5b194b31ff3bd34feee86ed634d46fa46fb14a62d743c8c

README.md => gitoid:blob:sha256:6fb9755a801d321ab42084fe169c131fddef5fc1cfda19ab4b6d32c2ec8fff7a

test.py => gitoid:blob:sha256:2cbbfad63f991b5199e78c543332ccc09fb5b44ccfc825b27af34a2523203b24

- The CLI does not interact with build tools directly. It works on files after they are compiled.
 - Does not track an entire project or store dependency relationships.
- The official documentation acknowledges that the CLI is a work-in-progress. Specifically, it notes that while the tool currently supports operations related to Artifact IDs, functionalities pertaining to manifests are not yet implemented. This aligns with the current state of the CLI, where commands and features related to input manifests are either incomplete or absent.

WEEK 6

SBOM Consumption: Vulnerability Exploitability eXchange

Workflow:

Generate SBOM

Import SBOM into the VEX generation tool and it should automatically scan for known vulnerabilities using some database.

The VEX generation tool can fetch public VEX documents if they have been uploaded by upstream vendors or users can provide VEX documents directly for the tool to ingest from a vendor or users can manually annotate the vulnerabilities and generate a VEX document themselves.

The VEX entries update users view of an SBOM by showing all risks. It gives you an enriched version of the SBOM with vulnerability data, VEX context and risk classifications to view.

```
{
  "vulnerabilities": [
    {
      "id": "CVE-2018-7489",
      "source": {
        "name": "NVD",
        "url": "https://nvd.nist.gov/vuln/detail/CVE-2019-9997"
      },
      "analysis": {
        "state": "not_affected",
        "justification": "code_not_reachable",
        "response": [
          "will_not_fix",
          "update"
        ],
        "detail": "An optional explanation of why the application is not affected by the vulnerable component."
      },
      "affects": [
        {
          "ref": "urn:cdx:3e671687-395b-41f5-a30f-a58921a69b79/1#jackson-databind-2.8.0"
        }
      ]
    }
  ]
}
```


<https://github.com/CycloneDX/bom-examples/blob/master/VEX/Use-Cases/Case-4/vex.json>

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.4",
  "version": 1,
  "metadata": {
    "timestamp": "2022-01-13T00:00:00Z",
    "component": {
      "name": "Acme Product",
      "type": "application",
      "bom-ref": "acme-product"
    }
  },
  "vulnerabilities": [
    {
      "id": "CVE-2020-25649",
      "source": {
        "name": "NVD",
        "url": "https://nvd.nist.gov/vuln/detail/CVE-2020-25649"
      },
      "ratings": [
        {
          "source": {
            "name": "NVD",
            "url":
https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator?vector=AV:N/AC:L/PR:
N/UI:N/S:U/C:N/I:H/A:N&version=3.1
          },
          "score": 7.5,
          "severity": "high",
          "method": "CVSSv31",
          "vector": "AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N"
        }
      ],
    }
  ],
}
```

```
{
  "source": {
    "name": "Acme Inc",
    "url":
https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator?vector=AV:N/AC:L/PR:
N/UI:N/S:U/C:N/I:H/A:N/CR:X/IR:X/AR:X/MAV:X/MAC:X/MPR:X/MUI:X/MS:X/MC:N/MI:N/MA:N
  },
  "score": 0.0,
  "severity": "none",
  "method": "CVSSv31",
  "vector":
"AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N/CR:X/IR:X/AR:X/MAV:X/MAC:X/MP
R:X/MUI:X/MS:X/MC:N/MI:N/MA:N"
  },
  "analysis": {
    "state": "not_affected",
    "justification": "code_not_reachable",
    "response": ["will_not_fix", "update"],
    "detail": "Automated dataflow analysis and manual code review
indicates that the vulnerable code is not reachable, either directly or
indirectly."
  },
  "affects": [
    {
      "ref": "acme-product",
      "versions": [
        {
          "range": "vers:semver/>=2.2.0|<=2.4.0",
          "status": "unaffected"
        }
      ]
    }
  ],
}
```

WEEK 7

10 March 2025

Impacts of SBOM Generation on Vulnerability Detection

Wanted to test how the SBOM generation tool and SBOM formats affect the vulnerability detection for Docker images.

They took the 100 most downloaded docker images, for each image they picked up to 25 different versions, giving them 2313 images to start with.

Used Syft and Trivy to generate SBOMs in CycloneDX and SPDX formats (giving them 4 SBOM datasets) and then analyzed these SBOMs using Trivy, Gype and CVE-bin-tool, and this outputs a list of vulnerabilities detected in the SBOM. In this study they just counted how many CVEs were found per SBOM.

They compared tools to see the difference in number of vulnerabilities based on SBOM generation tool and SBOM format

To look at the statistical significance of the differences they used bootstrapping

To look at the practical significance of their findings they calculated Cohen's D values, this indicated the extent of the variability in vulnerability detection attributed to different SBOM generation tools.

Areas of exploration:

Investigating which dependencies are frequently missed by SBOM generation tools, or if any dependencies frequently occur as false positives. This can help us see which tools are riskier to rely on and whether and how tools can be layered for better coverage.

Exploration of inconsistencies caused by tool differences, and maybe building systems that use multiple tools together to get better, more reliable results.

additional research is needed to effectively generate SBOMs for legacy packages and repositories where the source code is missing

Whether NTIA-compliant SBOMs contribute to the overall security of the software supply chain management process